



RIOC 2025

TD – Usine de Trombone

Alexandre Létocart

Contexte : Vous construisez une API backend de Clipper, une simulation d'usine qui fabrique des trombones.

Stack : Java 25, Spring Boot 4.0.6, MySQL 8.4 (Docker), Lombok 1.18.46, MapStruct 1.6.3

PS : Le code (noms de classes, variables, endpoints) doit être rédigé en anglais.

1. Création de l'entité Usine

Champs attendus

CHAMP	TYPE	NOTES
id	Long	Clé primaire, auto-générée (IDENTITY)
name	String	Nom de l'usine
production	Integer	Nombre de trombones produits par cycle
stock	Integer	Stock actuel, défaut à 0
latitude	Double	Position géographique (utilisé au TD3)
longitude	Double	Position géographique (utilisé au TD3)

Vous êtes libre d'ajouter des champs.

2. Les différents endpoints

Vous devez livrer plusieurs endpoints :
Chaque endpoint doit retourner le code HTTP approprié.

MÉTHODE	URL	DESCRIPTION
POST	/api/factories	Créer une usine
GET	/api/factories	Lister toutes les usines
GET	/api/factories/{id}	Détail d'une usine
PUT	/api/factories/{id}	Modifier entièrement (remplacement)
PATCH	/api/factories/{id}	Modifier partiellement (champs non-null)
POST	/api/factories/{id}/produce	Produire manuellement (?quantity=N)
DEL	/api/factories/{id}	Supprimer une usine

3. Production automatique des trombones

Après avoir fait le CRUD (*Create, Read, Update, Delete*), chaque usine créer doit produire automatiquement des trombones toutes les 5 secondes. À chaque cycle, le stock de chaque usine augmente de son rythme de production. Cette production doit tourner en arrière-plan sans intervention du client.

Voici des outils qui peuvent servir pour cette automatisation :

- <https://www.baeldung.com/spring-scheduled-tasks>
- <https://www.baeldung.com/spring-scheduling-annotations>



RIOC 2025

TD – Magasin de Trombone

Alexandre Létocart

Contexte : Les usines produisent des trombones, il faut maintenant les acheminer vers des magasins. Un magasin peut commander un réapprovisionnement à une usine. Le transfert n'est pas instantané : les trombones voyagent à la vitesse du son (1235 km/h).

Prérequis : TD1 terminé.

1. Création de l'entité Magasin

Champs attendus

CHAMP	TYPE	NOTES
id	Long	Clé primaire, auto-générée (IDENTITY)
name	String	Nom du magasin
stock	Integer	Stock actuel, défaut à 0
latitude	Double	Position géographique (utilisé au TD3)
longitude	Double	Position géographique (utilisé au TD3)

Vous êtes libre d'ajouter des champs.

2. Réapprovisionnement

Un magasin peut consulter les usines disponibles et commander des trombones.

MÉTHODE	URL	DESCRIPTION
GET	/api/stores/{id}/factories	Lister les usines disponibles avec leur distance
POST	/api/stores/{id}/supply	Commander un réapprovisionnement
GET	/api/stores/{id}/shipments	Lister les expéditions (?status=delivered pour les terminées, en cours par défaut)

Règles :

- **GET** /*factories* retourne toutes les usines avec leur stock et une distance aléatoire (10–1000 km). La distance réelle sera calculée au TD3 via une API externe
- **POST** /*supply* reçoit un identifiant d'usine et une quantité. La réponse doit inclure les informations de localisation (nom, latitude, longitude) de l'usine et du magasin concernés
- Le stock de l'usine est décrémenté immédiatement
- Le stock du magasin augmente uniquement après le temps de livraison
- Si l'usine n'a pas assez de stock → erreur appropriée
- Temps de trajet = $(\text{distance} / 1235) \times 3600$ secondes
- **GET** /*shipments* retourne par défaut les expéditions en cours. Le query param ?status=delivered permet de consulter les expéditions terminées

Vous pouvez ajouter d'autres règles métier

3. Livraison asynchrone

Vous devez modéliser l'expédition en base (usine source, magasin destinataire, quantité, distance, statut, heures de départ/arrivée). Un processus en arrière-plan vérifie toutes les **5 secondes** si des expéditions ont atteint leur destination. Si oui, le stock du magasin est mis à jour.

4. Concurrency

La production automatique (TD1) et le réapprovisionnement modifient le stock de l'usine en parallèle. Deux écritures concurrentes ne doivent jamais écraser silencieusement les données de l'autre. En cas de conflit, la commande doit être réessayée automatiquement (max 3 tentatives).

Voici des outils qui peuvent servir :

- <https://www.baeldung.com/jpa-optimistic-locking>
- <https://www.baeldung.com/spring-programmatic-transaction-management>



RIOC 2025

TD – Magasin / Usine - Geocodage

Alexandre Létocart

1. Géocodage des adresses

Les usines et magasins sont désormais créés avec une adresse. La base de données ne stocke que latitude et longitude — jamais l'adresse.

- Écriture : le client envoie une adresse → le serveur la convertit en coordonnées via l'API → persiste uniquement latitude et longitude
- Lecture : le serveur lit latitude et longitude depuis la base → appelle l'API inverse → retourne l'adresse dans la réponse

API de géocodage

ENDPOINT	URL	DESCRIPTION
Géocodage	GET /search/?q={adresse}&limit=1	Adresse → coordonnées
Inverse	GET /reverse/?lat={lat}&lon={lon}&limit=1	Coordonnées → adresse

Base URL : <https://api-gouv.lab.rioc.fr>

Pour tester, essayer sur un navigateur : <https://api-gouv.lab.rioc.fr/search/?q=quai%20de%20la%20somme>

Et : <https://api-gouv.lab.rioc.fr/reverse/?lat=49.904008&lon=2.286525>

- <https://www.baeldung.com/spring-boot-restclient>

- <https://www.baeldung.com/spring-value-annotation>

3. Distance Réelle

Remplacez la distance aléatoire par la formule de haversine. C'est la distance à vol d'oiseau entre deux points GPS.

Implémentez le calcul manuellement, sans bibliothèque externe.

- Le réapprovisionnement (POST /supply) utilise la vraie distance usine → magasin
- Les usines disponibles (GET /factories) affichent la vraie distance par rapport au magasin

- <https://www.baeldung.com/java-find-distance-between-points>

3. Endpoint de distance

Exposez un endpoint qui calcule la distance entre deux adresses quelconques. Cet endpoint sera utilisé dans le TP.

User-generated artifact content		DESCRIPTION
GET	/api/geo/distance?from={adresse}&to={adresse}	Distance entre deux adresses
La réponse contient : l'adresse normalisée, les coordonnées et la distance en kilomètres pour chaque point.		